

Emwani Immersive Scroll Website Research

The core conclusion

Yes, you can build this. The blocker is not a lack of tools. The blocker is that the project still needs a **single motion architecture, a shared project memory, a reliable asset pipeline, and a clear separation between DOM storytelling, scroll-scrubbed media, and true realtime 3D**. Your own blueprints already identify the missing Product Assembly chapter, empty or placeholder motion assets, and unfinished motion folders; that means the problem is implementation discipline and asset readiness, not whether Gemini, Jules, Antigravity, Claude, or Cloud Code are “powerful enough.”

filecite turn0file0 filecite turn0file1

The best strategy is **hybrid, not full-WebGL everywhere**. Use standard HTML/CSS sections for most narrative chapters, GSAP ScrollTrigger for pinned/scrubbed storytelling, and reserve Three.js or React Three Fiber for one or two hero moments where you truly need camera movement, depth, or object interaction. GSAP officially supports linking animation progress to scroll with `scrub`, pinning elements during scroll, and building timeline-based sequences, while Three.js and React Three Fiber give you model loading, animation timing, instancing, and performance controls for the sections that actually need realtime 3D. [1]

Your current site direction already supports this incremental path. The uploaded `website.html` shows a cinematic hero layer, still-image chapters, a brewing video block, and poster-based placeholders for grinding and other motion sections. That means you do **not** need to throw the site away and restart from zero; you can progressively replace poster sections with scroll-scrubbed media and add the missing assembly chapter as a new pinned sequence. filecite turn0file2

What your references are actually asking for

From the reference tutorial video you shared, the “Product Assembly Effect” is not evidence that the whole site must be a continuously simulated 3D environment. It is closer to an **art-directed exploded-view transition**: start with a clean product photo, generate an exploded or deconstructed target image, then produce a short transition clip between the two. That is an important distinction, because it means your flagship effect can be produced as an authored sequence first, and only then embedded into the site as a scroll-scrubbed experience.

That matches your internal project documents. The audit blueprint explicitly flags the Product Assembly chapter as missing, lists multiple absent motion assets, and calls out performance and optimization issues in the old build. The Phase 6 motion blueprint then says the motion asset folders are empty and that required grinding and brewing clips cannot be integrated until those assets actually exist. In other words: the site is already “motion-ready” in structure, but not yet “motion-complete” in assets.

filecite turn0file0 filecite turn0file1

The right interpretation is this: **first author the motion asset**, then **attach it to scroll**, then **layer in a realtime 3D section only where it adds real value**. If you reverse that order, you will spend weeks building a complicated WebGL shell around an effect that could have been produced in a far simpler pipeline.

Why the current project still feels stuck

The first reason is that there is no durable “project brain” yet. Gemini CLI’s official docs recommend repo-level and hierarchical `GEMINI.md` files so the model can load project instructions, coding style, and task context on every prompt, and Gemini CLI also exposes `/memory` commands to inspect and refresh those context files. Jules has a similar pattern with `AGENTS.md`, which it automatically checks in the repository root to understand how your codebase and agents are meant to work. Without those shared files, every tool is smart in isolation but forgetful in practice. [2]

The second reason is that the motion system has not been decomposed into layers. The web platform already gives you the right primitives: Intersection Observer for visibility-based loading and animation gating, `requestVideoFrameCallback()` for efficient per-frame video operations, and GSAP ScrollTrigger for mapping scroll progress to scrubbed or pinned timelines. If those are not assigned clear roles, teams often mix looping videos, poster fallbacks, image sequences, and 3D canvases in an ad hoc way, which makes performance and debugging much worse. [3]

The third reason is performance strategy. React Three Fiber’s docs are very explicit that WebGL is expensive, that on-demand rendering can reduce idle rendering costs, and that you should watch draw calls, reuse assets, use loader caching, and scale quality when performance drops. Three.js likewise documents `InstancedMesh` as the route for reducing draw calls when many repeated objects share geometry and materials. If you try to make every section “live 3D” instead of selectively using it, you will create battery, thermal, and mobile problems for no real storytelling gain. [4]

The fourth reason is that the production tools are being treated as interchangeable. They are not. Gemini CLI is strongest as a repo-aware planner and terminal agent. Jules is strongest as an asynchronous implementation and PR machine on top of GitHub-connected repos. Cloud Code is strongest as a deploy/debug/cloud-development surface for Cloud Run and Kubernetes, not as your primary cinematic design tool. Your own screenshots also show that Jules in your account is already oriented around repo configuration, proactive suggestions, skill-based agents, Render integration, CLI access, and an API surface, which reinforces that it should be used as an implementation/execution layer rather than as the place where creative direction is invented.

The architecture I would use for this site

Use a layered motion stack

Use plain HTML/CSS for typography, layout, copy flow, and product cards. Use GSAP ScrollTrigger for chapter pinning, scrubbed progress, staggered reveals, and section

timing. Use video or image sequences for “authored” motion moments like the Product Assembly effect. Use Three.js or React Three Fiber only for sections that genuinely benefit from camera-driven depth, such as a hero object orbit, a floating-parts reveal, or cursor-reactive parallax. This pattern is the cleanest fit with your current uploaded website and with the official capabilities of ScrollTrigger, Three.js, and React Three Fiber. [filecite](#)[turn0file2](#) [\[5\]](#)

Treat the Product Assembly effect as authored media first

For the assembly reveal itself, there are two sound implementations.

If you want **perfect frame accuracy**, use an **image sequence**. ScrollTrigger drives an integer frame index, and your code swaps rendered images or canvased frames. This is the safest route for an art-directed premium moment because it behaves like a deterministic timeline tied directly to scroll progress. GSAP’s scrub model is made for this kind of scroll-linked playhead control. [\[6\]](#)

If you want **lighter authoring and smaller asset management**, use a **short video scrub**. Control `video.currentTime` from ScrollTrigger progress and use `requestVideoFrameCallback()` when you need to paint or synchronize the current frame more precisely. MDN documents that `requestVideoFrameCallback()` fires as new video frames are sent to the compositor and is intended for efficient per-frame video operations. [\[7\]](#)

My recommendation for Emwani is straightforward: use an **image sequence for the Product Assembly chapter** and use **regular muted looping videos** for ambient chapters like brewing, grinding, smoke, or café atmosphere. That gives you exact hero control where it matters and lighter implementation everywhere else.

Keep true 3D narrow and intentional

If you want “real depth, real motion, and interactive elements,” do that with a **single hero 3D scene** or a **single product section**, not the whole site. Three.js supports loading glTF 2.0 assets via `GLTFLoader`, and the loader supports common performance-oriented extensions including compression and GPU instancing-related extensions. `AnimationMixer` lets you jump to an exact time with `setTime()`, which is useful when scroll is driving a model animation rather than letting it autoplay. React Three Fiber adds control over rendering cadence with `frameLoop="demand"` and quality scaling guidance through its performance documentation. [\[8\]](#)

That means your “true 3D” section should probably be one of these:

A premium hero with a slowly rotating pack, specular highlights, and cursor-reactive micro-parallax.

A product assembly moment where the product exists as a glTF model and scroll drives the mixer time.

A limited ambient object field, such as floating beans or packaging rings, built with instancing instead of individual meshes. Three.js documents `InstancedMesh` specifically as a way to reduce draw calls when many copies share geometry and materials. [9]

Build a shared memory layer before writing more code

Create these repo files immediately:

`GEMINI.md`

Global project instructions for Gemini CLI. Gemini CLI loads `GEMINI.md` hierarchically from user, workspace, and just-in-time directories, and includes those instructions with prompts. [10]

`AGENTS.md`

Repo instructions for Jules. Jules explicitly checks for `AGENTS.md` in the repo root to understand your agents, tools, and conventions. [11]

`docs/motion-spec.md`

Chapter-by-chapter timing, pin durations, triggers, scroll distances, and mobile fallbacks.

`docs/brand-motion.md`

Lighting, camera, typography, colors, pacing, and “never do this” rules.

`docs/performance-budget.md`

Maximum hero asset weights, maximum active canvases, target draw-call ceilings, mobile substitutions, and reduced-motion rules.

`docs/asset-pipeline.md`

How raw photos become exploded stills, how stills become video or sequences, naming conventions, export dimensions, and poster generation.

Once those exist, all of your tools stop guessing and start collaborating around the same memory.

The best workflow for each tool you have

Gemini CLI

Gemini CLI should be your **planning, repo-reading, codebase analysis, and implementation-spec tool**. Official docs show that it supports project context via `GEMINI.md`, built-in task planning with todos, Plan Mode as a read-only planning environment, web search and fetch, headless mode for scripting, and detailed session and model stats. It also supports checkpointing via settings and recovery via `/restore`, plus `/rewind` for stepping back in conversation and AI-made file changes. [12]

Use Gemini CLI for:

- repo-wide audit of the current front-end
- planning the chapter architecture before code edits
- generating the exact file list and diff plan

- producing a motion spec from your screenshots and blueprints
- writing helper scripts for sequence generation, poster export, or lazy-loading
- CI-friendly automation through headless mode

If you stay in Gemini CLI for too long on one task, use `/compress` to reduce context cost and `/stats` to monitor usage. If you use API-key auth rather than OAuth, token caching can reduce repeated-context cost; the docs note that token caching is available for API key and Vertex AI setups, but not for OAuth-based personal or enterprise Code Assist accounts. [13]

Jules and Jules CLI

Jules should be your **asynchronous coding worker**. Official docs say it integrates with GitHub, clones your repo into a cloud VM, installs dependencies, proposes a plan before writing code, lets you review diffs, and then creates a branch or PR workflow from the results. Jules also accepts image uploads during task creation, supports repo-level `AGENTS.md`, has setup scripts and reusable environment snapshots, and offers both a web UI and a CLI/API surface. [14]

Use Jules for:

- implementing the scroll-scrub component after the spec is approved
- converting placeholder sections into real motion sections
- adding lazy loading, poster fallbacks, and intersection-based play/pause
- generating PRs for performance cleanup
- running parallel tasks for design polish, security cleanup, and motion QA
- maintaining background improvement tasks on the repo

The official plan table shows that Jules and its higher tiers differ mainly in throughput and concurrency, not in whether they can do real work; the paid tiers simply let you run many more tasks in parallel and access newer models more aggressively. [15]

Antigravity

Based on your screenshot, Antigravity looks like a **multi-workspace, agent-oriented IDE surface** with an Agent Manager, repo cloning, and an editor-plus-agent workflow. I was not able to verify public official documentation for its current feature set in this pass, so I would treat it as an **orchestration shell** rather than the source of truth.

Use Antigravity for:

- running multiple agents against the same project from different workspaces
- keeping a design-review agent and an implementation agent separate
- collecting screenshots, tasks, and browser-test artifacts
- supervising the work produced by Gemini CLI or Claude/Jules prompts

That means Antigravity can be the cockpit, but your durable truth should still live in repo files like `GEMINI.md`, `AGENTS.md`, and your spec documents.

Claude Code, Claude Design, and Claude desktop surfaces

From your screenshots, your Claude account includes Usage, Capabilities, Connectors, Claude Code, Cowork, Claude in Chrome, and visible Claude Design usage status. I did not independently verify current public docs for those exact surfaces in this research pass, so I would use them in a **clearly scoped way**:

Use Claude Code for code review, code refactors, and alternate implementation ideas after Gemini or Jules has produced a plan.

Use Claude Design for visual systems work: hierarchy, pacing, light direction, moodboards, micro-interaction rules, hero compositions, and “what should this feel like” decisions.

Use Claude desktop/chat as the high-level strategic editor that compares outputs from Gemini and Jules and rewrites poor prompts into better ones.

In practical terms, Claude should be your **creative critic and second opinion**, not the only execution engine.

Cloud Code

If by CloudCode you mean Google Cloud Code, the official docs position it as IDE support across the development lifecycle for Kubernetes and Cloud Run applications, with Cloud Run support, templates, debugging, and bundled Gemini Code Assist integration for VS Code, IntelliJ, and Cloud Shell. That makes it useful for **previewing, debugging, and deploying** the final site if you are hosting on Google Cloud, but it is not your best primary tool for visual motion R&D. [\[16\]](#)

Use Cloud Code for:

- deployment and environment sanity
- Cloud Run previews
- debugging production config
- pushing the completed front-end to a managed runtime

Do not make Cloud Code the place where you invent the cinematic experience. Make it the place where you ship it cleanly.

The prompt library I would actually use

The master build prompt for Gemini CLI

Paste this into Gemini CLI after you create `GEMINI.md` and before you allow code edits:

You are the lead creative technologist for the Emwani M1 immersive scroll website.

Project goal:

Create a premium scroll-driven story website with real depth, controlled

motion, and interactive sections. We need one flagship Product Assembly chapter, one scroll-driven video chapter, and one optional realtime 3D hero/product scene.

Non-negotiables:

- Preserve the brand tone: premium, cinematic, African luxury, editorial restraint.
- Do not redesign the product cards unless required for motion integration.
- Prefer progressive enhancement over full rebuild.
- Desktop gets the full effect; mobile gets lighter motion and reduced-motion fallbacks.
- Accessibility and performance are mandatory.
- No uncontrolled autoplay chaos. Motion must feel deliberate.

Technical direction:

- Use DOM/CSS/GSAP ScrollTrigger for most sections.
- Recommend image-sequence scrubbing for the Product Assembly chapter.
- Recommend muted looping video with IntersectionObserver for ambient sections.
- Use Three.js / React Three Fiber only if a specific section truly needs realtime depth.
- Add lazy loading, poster fallbacks, and reduced-motion behavior.

What I want first:

1. Audit the current repo structure.
2. Produce a chapter-by-chapter motion architecture.
3. List all new files to create and all existing files to modify.
4. Write a phased implementation plan.
5. Do not modify code yet. Make a plan first.
6. Then create todos and wait for approval.

This prompt fits Gemini CLI's documented strengths: Plan Mode, todo-based task planning, repo context through GEMINI.md, and web/documentation research when needed. [17]

The implementation prompt for Jules

Use this in Jules after selecting the repo and branch, and attach your screenshots or mockups during task creation because Jules supports PNG/JPEG uploads up to 5MB at task start. [18]

Implement a premium Product Assembly scroll chapter for this website.

Requirements:

- Add a new section between the existing product-narrative chapters and the products grid.
- Desktop behavior: pin the section while the user scrolls through an assembly sequence.
- Use an image sequence for precise frame-by-frame scroll syncing.
- Create poster fallback and low-motion fallback.

- Mobile behavior: no pin, simplified autoplay-on-enter or scrub-lite sequence.
- Respect prefers-reduced-motion.
- Keep all edits localized and avoid changing unrelated product card styling.
- Add comments explaining the sequence loader, frame preload strategy, and ScrollTrigger math.
- Add a basic performance guard so we do not preload the full sequence all at once on weak devices.
- After coding, summarize changed files, created files, and follow-up recommendations.
- Before writing code, show me the full plan and assumptions.

This matches Jules' documented repo/branch flow, plan-before-code behavior, VM execution model, diff review, and branch/PR workflow. [\[19\]](#)

The design-direction prompt for Claude Design

Act as the design director for a premium cinematic coffee brand website.

I need a motion language for an immersive scroll experience with these rules:

- cinematic, elegant, editorial, premium
- no gimmicky cyberpunk motion
- black / bone / gold / deep coffee palette
- motion should feel expensive, not loud
- the Product Assembly chapter should feel engineered and poetic
- the hero should feel atmospheric and spacious
- the brewing section should feel warm and tactile
- all motion must degrade gracefully on mobile

Give me:

- motion principles
- camera language
- lighting language
- typography behavior during scroll
- transition pacing rules
- forbidden visual mistakes
- specific recommendations for the Product Assembly effect

The product-assembly still prompt for image generation

Use this in whichever image-generation surface gives you the best product consistency.

Create a luxury commercial exploded-view hero image of [PRODUCT NAME].

Goal:

A premium assembly reveal for a cinematic website section.

Source constraints:

- preserve the exact silhouette, proportions, branding zones, label placement, cap geometry, and material identity of the original product
- do not invent new parts

- do not distort or melt the packaging
- all separated pieces must remain aligned on the natural product axis
- components should feel physically believable and mechanically organized

Art direction:

- editorial product photography
- dark premium studio background
- controlled rim lighting
- subtle warm highlights
- realistic shadows and internal reflections
- macro detail, sharp focus on the product
- restrained cinematic depth of field
- luxury commercial look, not sci-fi fantasy

Composition:

- centered product
- exploded parts floating in a clean vertical sequence
- generous negative space for headline copy
- no extra text
- no watermark
- no hands
- no clutter

Output:

- one polished final frame intended to become the ending frame of a short assembly reveal video

The start-to-end video prompt for the assembly transition

This is the critical one for recreating the reference tutorial effect.

Create a short premium commercial transition from the original product photo to the exploded assembly target image.

Start frame:

the clean intact product

End frame:

the exploded assembly composition

Motion direction:

- slow controlled deconstruction
- parts separate smoothly along the real assembly axis
- micro-rotations only where physically plausible
- no chaotic spinning
- no camera whip
- no elastic or cartoony motion
- preserve brand geometry throughout
- maintain premium studio lighting and consistent perspective
- keep the background dark and elegant

Style:

- luxury product ad
- tactile materials
- precise engineering feel
- cinematic but restrained

Output goal:

a short sequence suitable for scroll-scrubbed web playback, where each frame feels art-directed and usable as a still

The performance and polish prompt for a code-review agent

Use this in Gemini CLI, Jules, or Claude Code after the feature exists.

Review this immersive scroll implementation as a senior frontend performance engineer.

Check:

- unnecessary layout thrashing
- oversized image sequence strategy
- eager preloading issues
- ScrollTrigger pin durations and cleanup
- video/canvas sync correctness
- reduced-motion behavior
- mobile fallback quality
- event listener leaks
- overuse of will-change
- GPU-heavy effects that can be simplified
- opportunities to lazy-load scenes and media

Return:

- top critical risks
- exact code changes to fix them
- a severity rating for each issue
- what to change now vs later

What I would do next in practice

Start by creating the shared project brain: GEMINI.md, AGENTS.md, docs/motion-spec.md, docs/brand-motion.md, and docs/performance-budget.md. Gemini CLI and Jules both have official support for repo-level context mechanisms, and this is the single biggest multiplier for consistency across all your tools. [20]

Then author the Product Assembly asset outside the website. Use your reference workflow: clean product photo, exploded still, short transition video, then convert that to either a scrubbed video or an image sequence. Do not start with live 3D unless you truly need a manipulable model.

Then implement only one flagship sequence first. The right first milestone is not “a whole fully 3D site.” The right first milestone is this: **one pinned Product Assembly chapter that feels premium, scrubs perfectly on desktop, and falls back gracefully on mobile**. Once that works, the rest of the motion system becomes much easier.

After that, add one purposeful 3D scene only if the result will genuinely improve the story. If you do, keep it inside a narrow performance budget: on-demand rendering where possible, cached loaders, compressed glTF assets, low draw calls, and instancing for repeated meshes. React Three Fiber and Three.js both document the primitives you need for that discipline. [21]

Finally, let Jules and your Claude/Gemini tools divide labor cleanly: Gemini plans, Jules implements, Claude critiques, Antigravity supervises, Cloud Code deploys.

Open questions and limitations

I could verify **Gemini CLI, Jules, Cloud Code, MDN browser APIs, GSAP, Three.js, and React Three Fiber** from official documentation in this pass. [22]

I could **not** independently verify current public official documentation for the exact Antigravity feature set or the specific Claude surfaces visible in your screenshots during this pass, so I treated them conservatively and used your screenshots only as evidence that those tools are available in your account.

“M1” is still ambiguous. I treated it mainly as your project/site label. If you instead meant Apple Silicon M1 hardware, the architecture above still holds, because it explicitly avoids turning the whole site into a permanently running WebGL app.

[1] [5] [6] <https://gsap.com/docs/v3/Plugins/ScrollTrigger/>

<https://gsap.com/docs/v3/Plugins/ScrollTrigger/>

[2] [10] [20] <https://geminicli.com/docs/cli/gemini-md>

<https://geminicli.com/docs/cli/gemini-md>

[3] https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

[4] [21] <https://r3f.docs.pmnd.rs/advanced/scaling-performance>

<https://r3f.docs.pmnd.rs/advanced/scaling-performance>

[7] <https://developer.mozilla.org/en-US/docs/Web/API/HTMLVideoElement/requestVideoFrameCallback>

<https://developer.mozilla.org/en-US/docs/Web/API/HTMLVideoElement/requestVideoFrameCallback>

[8] <https://threejs.org/docs/pages/GLTFLoader.html>
<https://threejs.org/docs/pages/GLTFLoader.html>

[9] <https://threejs.org/docs/pages/InstancedMesh.html>
<https://threejs.org/docs/pages/InstancedMesh.html>

[11] [14] [19] <https://jules.google/docs>
<https://jules.google/docs>

[12] [22] <https://geminicli.com/docs/>
<https://geminicli.com/docs/>

[13] <https://geminicli.com/docs/reference/commands/>
<https://geminicli.com/docs/reference/commands/>

[15] <https://jules.google/docs/usage-limits/>
<https://jules.google/docs/usage-limits/>

[16] <https://cloud.google.com/code/docs>
<https://cloud.google.com/code/docs>

[17] <https://geminicli.com/docs/cli/plan-mode>
<https://geminicli.com/docs/cli/plan-mode>

[18] <https://jules.google/docs/running-tasks/>
<https://jules.google/docs/running-tasks/>